

Planning Logic

The build order follows a strict dependency chain, since later components rely on earlier ones being in place:

1. **Requirement analysis before configuration** — all business requirements, field dependencies, validation rules, and expected behaviors are identified before development begins. This ensures that each configuration component directly addresses a defined requirement.
2. **UI Policy must exist before its actions** — the **High Impact Control** UI Policy is created first, because UI Policy Actions attach to it as child records.
3. **UI Policy Actions before Client Scripts** — mandatory/read-only behavior is declarative and independent of the scripts, so it is configured next and can be verified on its own.
4. **Client-side behavior is separated by responsibility** — each Client Script handles one specific requirement rather than combining multiple behaviors into a single script. This makes troubleshooting, maintenance, and testing easier.
5. **onChange script before onSubmit script** — auto-populating Urgency through the **onChange** script happens earlier in the user's workflow than the save-time validation performed by the **onSubmit** script. Building and

testing them in this order prevents confusion between automatic field population and final form validation.

6. **Validation rules are designed to prevent invalid data rather than correct it later** — wherever possible, incorrect values are blocked or corrected at the point of entry. This improves data quality and reduces the need for manual cleanup.
7. **onCellEdit script last, and tested separately** — it operates on a different surface (list view rather than the form) and has no dependency on the other components. It is therefore isolated at the end to avoid interference during form-level testing.
8. **Server-side and client-side responsibilities are considered separately** — client-side controls improve the user experience, while important data-integrity requirements should also be considered for enforcement at the appropriate server-side level where required.
9. **Error handling is planned along with the functionality** — validation failures and invalid state changes should provide clear feedback to the user so that the required corrective action is immediately understandable.
10. **Each component is independently verifiable** — after configuration, the relevant behavior is checked before moving to the next dependent component. This helps identify configuration errors early and reduc

